
LAB 6 ASSIGNMENT

Docker & Microservices – Decomposing and Running the Backend as Independent Services

Web Services & SOA Laboratory • Lab 6 • Microservices

Important: Continue from the Dockerized application completed in the earlier lab work. Decompose the backend into independently runnable services, containerize them, connect them through a Docker network, and demonstrate service-to-service communication.

1. Objective

Extend the Dockerized CampusConnect backend into a simple microservices-based application. Identify meaningful service boundaries, separate the application into independently runnable services, expose REST endpoints, configure service-to-service communication using Docker networking and environment variables, and orchestrate the complete application with Docker Compose.

2. Learning Outcomes

- Explain why an application may be decomposed into multiple services.
- Identify reasonable service boundaries in the existing application.
- Implement at least three independently runnable REST services: User, Product and Order.
- Containerize each service using a suitable Dockerfile.
- Connect services through a Docker network without using localhost for container-to-container communication.
- Configure service URLs and ports using environment variables.
- Use Docker Compose to run the complete multi-service application.
- Test individual services and service-to-service communication using Postman.
- Document the final microservices architecture, endpoints, configuration and testing evidence.

3. Lab Context

Earlier application work: Frontend, REST APIs, backend and database persistence were developed in previous labs.

Lab 6: Decompose the backend into multiple services, containerize them, connect them through Docker networking, and run the complete application with Docker Compose.

4. Starting Point and Requirements

Do not rebuild the application from scratch. Reuse the working application and Docker concepts already completed.

Minimum recommended decomposition: User Service, Product Service and Order Service. Each service must have its own source code, REST endpoints, Dockerfile, configuration and container.

5. Service Boundaries

Each service should have one clear responsibility and remain independently runnable.

Service	Responsibility	Example endpoints
User Service	Create, retrieve, update and delete User resources.	GET /users GET /users/{id} POST /users PUT /users/{id} DELETE /users/{id}
Product Service	Manage Product resources used by the application.	GET /products GET /products/{id} POST /products PUT /products/{id} DELETE /products/{id}
Order Service	Create and retrieve Orders; validate referenced User and Product data through APIs.	POST /orders GET /orders GET /orders/{id}

6. Architecture

Client / Postman		API Gateway Optional concept
↓		↓
User Service :3001	Product Service :3002	Order Service :3003
User DB owned by User	Product DB owned by Product	Order DB owned by Order

Order Service communicates with User and Product through REST APIs over the shared Docker network.

7. Database per Service

Each service owns its data. Other services must not directly query another service's database.

Rule: Order Service → REST/API → User Service, rather than Order Service → User database.

8. Service-to-Service Communication

Example: Order Service validates an order by requesting User and Product information.

Order Service → GET http://user-service:3001/users/101 → User Service

Order Service → GET http://product-service:3002/products/501 → Product Service

Inside Docker, use the Compose service/container name as the hostname. Do not use localhost to refer to another container.

Verification

- The calling service successfully receives a response from the target service.
- The architecture diagram, Compose file and service URLs describe the same implementation.

9. API Gateway Concept

A gateway can provide one client-facing entry point and route requests to internal services.

Client React / Postman	API Gateway one entry point	Internal services /users • /products • /orders
→	→	

For Lab 6, the gateway is a conceptual routing example; detailed gateway implementation may be handled later.

10. Inter-Service Error Handling

When a dependency is unavailable, the calling service should fail clearly and gracefully rather than hanging or returning an unclear error.

Order Service → User Service (unavailable) → 503 Service Unavailable

Timeout Stop waiting after a limit	Retry Limited repeat attempt	Fallback Safe alternative response	Circuit breaker Concept for repeated failure
---------------------------------------	---------------------------------	---------------------------------------	---

These are concepts for this lab; detailed resilience implementation is not required here.

11. Containerize the Microservices

Each application service must be packaged as its own Docker image.

Step 1 — Create a Dockerfile for Each Service

- Create a Dockerfile in each service directory.
- Use an appropriate base image, working directory and dependency installation.
- Copy the service source code.
- Expose the service port and define the startup command.

Example Dockerfile

```
FROM node:22
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3001
CMD ["node", "server.js"]
```

Verification: each service can be built into a separate Docker image.

12. Build Images and Create the Shared Network

```
docker build -t user-service:v1 ./user-service
docker build -t product-service:v1 ./product-service
docker build -t order-service:v1 ./order-service
docker images
```

Use meaningful image names and tags.

Shared Docker Network

```
docker network create campus-network
```

- Attach all application services to the same Docker network.
- Use service/container names for internal communication.
- Keep host port mappings only where external access is required, such as Postman.

Environment Variables

- Do not hard-code service URLs or environment-specific ports.

```
USER_SERVICE_PORT=3001
PRODUCT_SERVICE_PORT=3002
ORDER_SERVICE_PORT=3003
USER_SERVICE_URL=http://user-service:3001
PRODUCT_SERVICE_URL=http://product-service:3002
```

Verification: change a port or target URL through configuration without changing application source code.

13. Docker Compose — Complete Application

```
services:
  user-service:
    build: ./user-service
    ports: ["3001:3001"]
  product-service:
    build: ./product-service
    ports: ["3002:3002"]
  order-service:
    build: ./order-service
    ports: ["3003:3003"]
  environment:
    USER_SERVICE_URL: http://user-service:3001
    PRODUCT_SERVICE_URL: http://product-service:3002
networks:
  campus-network:
```

Modify the example to match the actual project structure and database configuration used.

14. Run and Verify with Docker Compose

```
docker compose config
docker compose up -d
docker compose ps
docker compose logs
```

- Confirm all application containers are running.
- Check logs for startup, networking or database connection errors.
- Confirm required database services and volumes are running.

15. Postman Testing Requirements

- Create a collection named Microservices – Lab 6.
- Test direct User, Product and Order endpoints.
- Test at least one real Order → User operation.
- Test at least one real Order → Product operation.
- Test an invalid resource ID and record the 404 response.
- Stop a dependency and record the controlled error/503 response.
- Restart the dependency and verify recovery.

Expected testing flow

Test	Expected
GET /users	200
GET /products	200
POST /orders	201
Order → User / Product	Successful dependency calls
Invalid User ID	404
Stop User → create Order	Controlled error / 503
Restart User → create Order	Success / recovery

16. Microservices API and Communication Exercise

Complete the following design table using your actual service decomposition.

Item	User Service	Product Service	Order Service
Responsibility			
Port	:3001	:3002	:3003
Main resources	Users	Products	Orders
Key endpoints	GET/POST/PUT/DELETE /users	GET/POST/PUT/DELETE /products	POST/GET /orders
Data/database	User-owned	Product-owned	Order-owned

Service-to-Service Call

Field	Your design
Calling service	Order Service
Target service	User Service / Product Service
HTTP method	GET
Endpoint	/users/{id} or /products/{id}
Request data	Referenced user/product ID
Expected response	200 + requested resource
Failure response	404 for invalid resource; 503 when dependency is unavailable

17. Required Architecture, Evidence and Submission

Architecture diagram must show:

- Client/Postman.
- Optional API Gateway.
- User, Product and Order services.
- Docker network.
- Database/data ownership for each service.
- Exposed ports.
- Arrows showing service-to-service communication.

Evidence / screenshots should include:

- Working application before decomposition.
- Service-boundary design and final architecture diagram.
- Separate service/project structure.
- Dockerfile for each service.
- Successful image builds and docker images output.
- Docker network showing service containers.
- Environment variables containing service URLs/ports.
- docker compose up -d and docker compose ps output.
- Startup logs.
- Postman tests for User, Product and Order.
- Order → User and Order → Product communication evidence.
- Database persistence/ownership evidence.
- Invalid ID 404 test and dependency-unavailable 503 test.

Submission files

- Source code for all three microservices.
- Dockerfile for each service.
- compose.yaml or docker-compose.yml.
- .dockerignore file(s) where appropriate.
- Environment-variable configuration without secrets.
- Postman collection and screenshots.
- Final architecture diagram.
- README.md.
- Completed API & Communication Exercise.

18. README.md Requirements

- Project overview and relationship to previous application work.
- Three service responsibilities and service boundaries.
- Service ports and endpoint mapping.
- How to run each service independently.
- Dockerfile explanation for each service.
- Docker image build commands.
- Docker network and service-name communication.
- Environment variables and service URLs.
- Database-per-service/data ownership approach.
- Docker Compose configuration and commands.
- Service-to-service communication flow.
- Inter-service error handling behaviour.
- Postman tests and expected responses.
- Troubleshooting issues and resolutions.

19. Hands-on Workflow

1. Start three services with Docker Compose
2. Verify all containers are running
3. Test User, Product and Order individually
4. Test Order → User / Product communication
5. Create an order using existing User/Product data
6. Stop User Service and observe the failure/503 behaviour
7. Restart User Service and verify recovery

20. Important Notes

- Keep each service focused, independently runnable and responsible for its own data.
- Use Docker service/container names for container-to-container communication.
- Keep ports and service URLs configurable; never hard-code secrets.
- The architecture diagram, Compose file, service URLs and screenshots must describe the same implementation.
- Gateway routing and resilience patterns are introduced conceptually here; detailed implementation can be covered later.

21. Final Checklist

- At least three independently runnable services created: User, Product and Order.
- Each service exposes and handles its own REST endpoints.
- Each service has a Dockerfile and image.
- Services communicate through a shared Docker network.
- Internal service URLs use service names, not localhost.
- Data ownership is documented and direct cross-service database access is avoided.
- Docker Compose starts the complete application.
- At least one real Order → User call works.
- At least one real Order → Product call works.
- Dependency failure produces a controlled error such as 503.
- Postman evidence covers direct and inter-service calls.
- Architecture diagram matches the implementation and is clearly readable.
- README documents complete setup, networking, testing and troubleshooting.

Target outcome: a working, testable, containerized microservices application.

End of Lab 6 Assignment